4.4.5 Bloom Filter: as Fast as Hash Tables, But Saving Memory (with a catch)

We haven’t officially met this data structure yet in the book, but there is a good chance you have already heard of *Bloom filters*. It is a data structure named after Burton Howard Bloom, who invented it in the 1970s.

There are three notable differences between hash tables and Bloom filters:

- Basic *Bloom filters* don't store data, they just answer the question: is a *datum* in the set? In other words, they implement *hash set*'s API, not *hash table’s*.
- Bloom filters require less memory in comparison to hash tables: this is the main reason for their use.
- While a negative answer is 100% accurate, there might be false positives: we will explain this in details in a few sections, for now just keep in mind that sometimes a Bloom filter might answer that a value was added to it, while it was not.
- It is not possible to delete a value from a Bloom filter[6].

There is a tradeoff between the accuracy of a Bloom filter, and the memory it uses. The less memory, the more false-positives it returns. Luckily, there is an exact formula that, given the number of values we need to store, can output the amount of memory needed to keep the rate of false positives within a certain threshold: we'll go into the details for this